

المكاتب المطلوبة من :

External_API:

composer install

Main APP:

composer install

طريقة التشغيل:

من مجلد External API قم بتشغيل:

```
php artisan serve --port=8001
```

من مجلد Main App قم بتشغيل ثلاثة طرفيات في نفس الوقت وفي كل طرفية اكتب التعليمة التالية على الترتيب:

```
php artisan serve
```

```
php artisan queue:work
```

```
artisan reverb:start
```

تفاصيل الطلبات:

1. Create Violation

- **Method & URL:** POST /api/system/violations
- **Description:** This endpoint is called by the AI system to register a new traffic violation. It validates the data, retrieves driver information from an external service, and stores the violation in the database.
- **Authentication:** Required (auth:sanctum). An authentication token must be sent.
- **Headers:**
 - Authorization: Bearer <YOUR_SANCTUM_TOKEN>
 - Accept: application/json
- **Request Body:**

JSON

```
{
  "violation_type_key": "red_light",
  "plate_number": "1234567",
  "camera_id": "CAM-001",
  "timestamp": "2025-08-04 18:30:00"
}
```

- **Success Response** (201 Created):

JSON

```
{
  "message": "تم تسجيل المخالفة بنجاح",
  "violation": {
    // ... violation details from the database
  },
  "driver_info": {
    // ... driver details from external API
  }
}
```

- **Error Responses:**

- 422 Unprocessable Entity: If validation fails (e.g., a required field is missing).
- 404 Not Found: If the plate_number does not exist in the external traffic system.
- 503 Service Unavailable: If the external traffic API cannot be reached.

2. Log Passing Car

- **Method & URL:** POST /api/system/passing-cars
- **Description:** Used by the AI system to log every time a car passes by a camera.
- **Authentication:** Required (auth:sanctum).
- **Headers:**
 - Authorization: Bearer <YOUR_SANCTUM_TOKEN>
 - Accept: application/json
- **Request Body:**

JSON

```
{
```

```
"plate_num": "1234567",
"camera_id": "CAM-001",
"timestamp": "2025-08-04 18:32:00"
}
```

- **Success Response** (201 Created):

JSON

```
{
  "id": 1,
  "plate_num": "1234567",
  "camera_id": "CAM-001",
  "timestamp": "2025-08-04T18:32:00.000000Z",
  "camera": {
    // ... camera details
  }
}
```

- **Error Response** (422 Unprocessable Entity): If validation fails (e.g., camera_id does not exist).
-

3. Create Accident

- **Method & URL:** POST /api/system/accidents
- **Description:** Used by the AI system to report a new accident.
- **Authentication:** Required (auth:sanctum).
- **Headers:**
 - Authorization: Bearer <YOUR_SANCTUM_TOKEN>
 - Accept: application/json
- **Request Body:**

JSON

```
{
  "camera_id": "CAM-002",
  "timestamp": "2025-08-04 18:35:00"
}
```

- **Success Response** (201 Created):

JSON

```
{
  "id": 1,
  "camera_id": "CAM-002",
  "timestamp": "2025-08-04T18:35:00.000000Z",
  "status": "new",
  "camera": {
    // ... camera details
  }
}
```

- **Error Response** (422 Unprocessable Entity): If validation fails.

4. Mark Accident as Viewed

- **Method & URL:** PATCH /api/admin/accidents/{accident}/viewed
- **Description:** Updates the status of a specific accident from new to viewed. This is typically called when a user views an accident notification in the admin panel.
- **Authentication:** Required (auth:sanctum and user must be an employee).
- **Headers:**
 - Authorization: Bearer <YOUR_SANCTUM_TOKEN>
 - Accept: application/json
- **URL Parameters:**
 - {accident}: The ID of the accident you want to update.
- **Request Body:** None.
- **Success Response** (200 OK): Returns the full, updated accident object.

JSON

```
{
  "id": 1,
  "camera_id": "CAM-002",
  "timestamp": "2025-08-04T18:35:00.000000Z",
  "status": "viewed",
  "camera": {
    // ... camera details
  }
}
```

}

- **Error Response** (404 Not Found): If the {accident} ID does not exist.
-

5. Search for Vehicle Sightings

- **Method & URL:** GET /api/admin/passing-cars/search/{plate_num}
- **Description:** Searches for all recorded sightings of a specific vehicle by its plate number and retrieves the driver's information.
- **Authentication:** Required (auth:sanctum and user must be an employee).
- **Headers:**
 - Authorization: Bearer <YOUR_SANCTUM_TOKEN>
 - Accept: application/json
- **URL Parameters:**
 - {plate_num}: The vehicle's plate number to search for.
- **Request Body:** None.
- **Success Response** (200 OK):

JSON

```
{
  "driver_info": {
    // ... driver details from external API
  },
  "sightings": [
    {
      "id": 1,
      "plate_num": "1234567",
      "camera_id": "CAM-001",
      "timestamp": "...",
      "camera": {
        // ... camera details
      }
    },
    // ... more sightings
  ]
}
```

- **Error Response (404 Not Found):** If the driver is not found in the external traffic system.

6. Get New Accidents

- **Method & URL:** GET /api/admin/accidents/new
- **Description:** This endpoint is used to fetch a list of all accidents that are still in a 'new' state (i.e., have not been viewed). It's typically used to populate an initial list of notifications when the user first loads the dashboard, before the real-time stream takes over for subsequent updates.
- **Authentication:** Required (auth:sanctum and user must be an employee).
- **Headers:**
 - Authorization: Bearer <YOUR_SANCTUM_TOKEN>
 - Accept: application/json
- **URL Parameters:** None.
- **Request Body:** None.
- **Success Response (200 OK):** Returns a JSON array of accident objects.

JSON

```
[
  {
    "id": 1,
    "camera_id": "CAM-002",
    "timestamp": "2025-08-04T18:35:00.000000Z",
    "status": "new",
    "camera": {
      // ... camera details
    }
  },
  {
    "id": 2,
    "camera_id": "CAM-005",
    "timestamp": "2025-08-04T19:00:00.000000Z",
    "status": "new",
    "camera": {
      // ... camera details
    }
  }
]
```

- **Error Response (401 Unauthorized):** If the request lacks valid authentication

7. Stream New Accidents

- **Method & URL:** GET /api/admin/accidents/stream
- **Description:** This request does not return data once and finish; instead, it establishes a persistent, open connection between the browser (Frontend) and the server (Backend) using a technology called Server-Sent Events (SSE). Its function is for the server to send new accident data to the dashboard in real-time as it occurs.
- **Authentication:** Required (auth:sanctum and user must be an employee).
- **Headers:**
 - Authorization: Bearer <YOUR_SANCTUM_TOKEN>
 - Accept: text/event-stream
- **URL Parameters:** None.
- **Request Body:** None.
- **Response:** The response is not a single JSON object but a stream of events. The server will continue to send an event named new-accident whenever a new accident is registered.
 - **Data Format:**
 - event: new-accident
 - data: {"id": 1, "camera_id": "CAM-002", "status": "new", ...}
 -
 - event: new-accident
 - data: {"id": 2, "camera_id": "CAM-005", "status": "new", ...}
- **Benefit for Frontend Developer:**
 1. **Real-time Updates:** Instead of the frontend having to ask for new accidents every few seconds (a process called Polling), this API reverses the logic. The browser opens a single connection and waits, and the server "pushes" new data the instant it happens. This provides immediate updates with high efficiency and low network overhead.
 2. **Better User Experience:** The user will see new accident notifications appear on the screen in the same moment they are reported, without needing to refresh the page. This makes the application feel dynamic and responsive.
 3. **Simpler Frontend Code:** Handling SSE in JavaScript is very straightforward using the native EventSource object built into modern browsers, making it easy to listen for these server-pushed event

